

ca and os topics

B21CA03DC — Computer Architecture & Operating Systems

Study topic map (Blocks 1–6, 24 units)

Learn each row on YouTube, Neso Academy, GeeksforGeeks, or your SLM PDF, then match it in CA and OS Note.md.

Item	Detail
Course	B21CA03DC
Programme	BCA, Semester II
University	SGOU
Full notes	CA and OS Note.md
SLM PDF	Computer Architecture and Operating System - B21CA03DC.pdf

How to use

1. Go **Block** → **Unit** in order (same as SGOU SLM).
2. For each table row, use the **Search as** column on any platform.
3. Read the matching section in your notes and try PYQs there.
4. Mark **Studied / Revised** dates in the progress table at the bottom.

Block 1: Basic Functional Architecture

Exam focus: generations table, von Neumann diagram, bus types, addressing modes with examples, instruction cycle phases.

Unit 1: Functional Units and Bus

SLM	Topic	Concepts to cover	Search as
1.1.1	Generations of computers	Five generations with time periods; switching tech (vacuum tubes → transistors → ICs → VLSI/microprocessors →	generations of computer history

SLM	Topic	Concepts to cover	Search as
		AI); size, cost, speed trends; name 2–3 machines per generation (ENIAC, UNIVAC, IBM 360, CRAY, PARAM).	
1.1.2	Architecture vs organisation	Architecture = what programmer sees (instruction set, data types, addressing, I/O). Organisation = how hardware implements it (units, buses, control paths). Same architecture can have different organisations.	computer architecture vs organisation
1.1.2	Von Neumann model	Stored-program idea: instructions and data in same memory. Five units linked by buses. Draw block diagram: Input ↔ Memory ↔ Output; CPU = Control + ALU + registers inside CPU.	von Neumann architecture diagram
1.1.2.1	Input unit	Accepts data/programs from outside; converts to binary/electrical form; examples keyboard (scan codes), mouse, scanner, camera; passes to memory/CPU.	input unit of computer
1.1.2.2	Memory unit	Primary: RAM (volatile, fast, run-time) and ROM (non-volatile, firmware/BIOS). Secondary: disks, optical — bulk, slower, cheaper. Programs must be in primary memory during execution; compare access time and cost.	primary vs secondary memory
1.1.2.3	ALU	Performs arithmetic (+, −, ×, ÷) and logic (AND, OR, NOT, compare). Operands from registers/memory; result back to register or memory. Part of CPU, not separate from control.	ALU arithmetic logic unit

SLM	Topic	Concepts to cover	Search as
1.1.2.4	Control unit	Fetches instruction from memory, decodes opcode, generates control signals and timing for all units; manages instruction sequencing (works with PC). "Nerve centre" of CPU.	control unit CPU
1.1.2.5	Output unit	Converts internal binary results to human-readable form; monitor, printer, speaker, plotter; opposite role of input unit.	output devices computer
1.1.3	Operational concepts	Instruction = opcode (operation) + operand(s) (data or address). Steps: fetch → decode → fetch operands → execute → store/write-back. Program Counter (PC) points to next instruction.	instruction cycle fetch decode execute
1.1.4–5	Bus structures	Bus = shared wires connecting units. Internal (system) bus links CPU, memory, I/O; external bus links peripherals. Single-bus structure: one common pathway (typical PC-style).	computer bus structure
1.1.4–5	System bus	Data bus (bi-directional, carries words). Address bus (uni-directional, selects location). Control bus (read/write, interrupt, clock, handshaking). Bus width affects bits per transfer and max addressable memory.	data address control bus
1.1.4–5	Dedicated vs multiplexed	Dedicated: separate wires for address and data always. Multiplexed: same lines used for address then data at different times (time multiplexing); needs control	multiplexed address data bus

SLM	Topic	Concepts to cover	Search as
		like Address Valid. Saves pins, adds timing complexity.	

Unit 2: Timing and Control

SLM	Topic	Concepts to cover	Search as
1.2.1	Digital building blocks	Flip-flops store 1 bit; registers = group of flip-flops; latches hold data; decoder selects one of many lines; clock synchronises all operations to edges/ticks.	flip flop register computer organization
1.2.2	Timing and control	Hardwired control: fixed logic gates generate control signals (fast, inflexible). Microprogrammed control: control signals from microinstructions in control memory (flexible, easier to change ISA).	hardwired vs microprogrammed control unit
1.2.2	Control unit timing	Each instruction broken into micro-operations (register transfer, memory read, ALU op). Control unit asserts signals in correct order and duration so data moves without conflict on the bus.	control signals timing computer
1.2.3	Sequence counter	Counter steps through states (T0, T1, ...) per instruction cycle; each state enables different micro-operations; generates timing signals for fetch, decode, execute phases.	sequence counter control unit

Unit 3: Addressing Modes

SLM	Topic	Concepts to cover	Search as
1.3.1	Need for addressing	Same opcode can reach operands in memory, registers,	why addressing modes needed

SLM	Topic	Concepts to cover	Search as
	modes	or inline constant — saves memory, supports loops, pointers, arrays, and structured data without rewriting code for every address.	
1.3.1	Implied (implicit)	Operand location is fixed by instruction (e.g. accumulator implicit); no extra address field in instruction; very compact encoding.	implicit addressing mode
1.3.1	Immediate	Operand value is inside the instruction itself (constant); fast; limited size of constant; e.g. ADD R1, #5.	immediate addressing mode example
1.3.1	Direct	Instruction holds actual memory address of operand; one memory access after fetch; simple but fixed address in code.	direct addressing mode
1.3.1	Register	Operand is in a named CPU register ; fastest access; no extra memory trip for address; common in RISC-style ISAs.	register addressing mode
1.3.1	Register indirect	Register holds address of operand in memory; good for pointers; one register + one memory access for data.	register indirect addressing
1.3.1	Auto inc/dec	Like register indirect, but register auto-increments or decrements after access — efficient for traversing arrays/strings.	auto increment addressing mode
1.3.1	Indirect	Instruction gives address of a memory location that holds the real operand address; two memory accesses; powerful but slower.	indirect addressing mode
1.3.1	Indexed	Effective address = base/displacement + index	indexed addressing mode

SLM	Topic	Concepts to cover	Search as
		register (often for arrays); index may scale by element size; base can be PC for relative jumps.	

Unit 4: Program Control

SLM	Topic	Concepts to cover	Search as
1.4.1	Program control overview	Instructions that change order of execution (not just next sequential PC+1): branches, calls, returns, traps; essential for loops, decisions, procedures.	program control instructions
1.4.2	Unconditional branch	Jump/branch always to target address; updates PC regardless of flags; used for loops end, goto, linking code sections.	unconditional branch instruction
1.4.3–4	Conditional branch	Compare sets condition flags (zero, carry, sign, overflow); branch only if condition true/false (BEQ, BNE, etc.); implements if-else and loop tests.	conditional branch instruction flags
1.4.5	Subroutines	Call saves return address (stack/link register), jumps to procedure; Return restores PC; enables reusable code; nested calls use stack.	subroutine call return stack
1.4.6–7	Halt & interrupt instr.	Halt stops program until reset/interrupt. Interrupt/trap instruction forces switch to ISR (software interrupt for OS services); differ from hardware IRQ timing.	halt interrupt instruction

Block 2: I/O and DMA

Exam focus: memory-mapped vs isolated I/O, polling vs interrupt, daisy chain, DMA diagram and modes.

Unit 1: Register Transfer Language (RTL)

SLM	Topic	Concepts to cover	Search as
2.1.1–2	RTL basics	Symbolic notation for register transfers (e.g. $R1 \leftarrow R2$, $M[addr] \leftarrow R1$); describes what happens each clock in abstract form; basis for micro-operations and control design.	register transfer language RTL
2.1.4	Micro-operations	Smallest steps: register load/clear, shift, ALU micro-op, memory read/write enable; one or more per clock; combined to implement full instructions.	microoperations computer
2.1.4	Bus/memory transfers	Read: place address, assert read, data arrives on data bus. Write: address + data + write signal. Must respect bus arbitration and timing.	memory read write microoperation

Unit 2: Input–Output Organization

SLM	Topic	Concepts to cover	Search as
2.2.1–2	Peripherals & ASCII	Keyboards, printers, disks as peripherals ; ASCII maps characters to 7/8-bit codes for text I/O; difference between character device and block device (concept).	ASCII in computer I/O
2.2.3	I/O interface	Interface adapts CPU/bus to device speed/protocol: data registers, status register (ready/busy/error), control register; buffer when CPU faster than device.	I/O interface computer
2.2.4–5	I/O bus & commands	CPU sends commands (read sector, print char) via interface;	I/O bus organization

SLM	Topic	Concepts to cover	Search as
		status polling or interrupts report completion; standardised bus (e.g. USB concept at high level).	
2.2.4–5	Isolated I/O	Separate I/O address space (ports); special IN/OUT instructions; memory addresses and I/O addresses distinct — typical in x86 “port mapped” style.	isolated I/O port mapped I/O
2.2.4–5	Memory- mapped I/O	Device registers appear at normal memory addresses ; same load/store instructions access device; simpler unified address map; no separate I/O instructions.	memory mapped I/O vs isolated I/O

Unit 3: Priority Interrupts

SLM	Topic	Concepts to cover	Search as
2.3.1–2	Interrupts	Asynchronous event stops CPU, saves context, runs ISR , returns to interrupted program. IRQ line, interrupt vector/table, interrupt acknowledge; faster than polling for rare events.	interrupt handling in computer
2.3.1–2	Interrupt types	Hardware (timer, disk, keyboard) vs software (trap/int instruction); maskable (can be disabled) vs non-maskable (critical faults); vector number picks ISR.	types of interrupts computer
2.3.3	Polling	CPU repeatedly reads status register in a loop; simple hardware; wastes CPU when events rare; OK for fast devices or embedded tight loops.	polling vs interrupt I/O
2.3.4–6	Priority schemes	When multiple IRQs pending, serve highest priority first. Daisy chain : grant propagates	daisy chain priority interrupt

SLM	Topic	Concepts to cover	Search as
		on bus. Parallel priority + encoder: hardware encodes highest active level.	

Unit 4: Direct Memory Access (DMA)

SLM	Topic	Concepts to cover	Search as
2.4.1-4	DMA concept	DMA controller transfers blocks between I/O device and memory without CPU moving each byte; CPU sets up transfer (source, dest, count), DMA takes bus, raises interrupt when done.	DMA direct memory access
2.4.1-4	DMA modes	Burst/block: hijacks bus for whole block. Cycle stealing: one word per bus cycle, CPU runs between steals. Transparent: DMA only when CPU not using bus. Compare CPU load and latency.	DMA burst mode cycle stealing
2.4.1-4	Why DMA	High throughput for disk/network/display; CPU free for other processes during bulk transfer; essential for modern multimedia and storage.	advantages of DMA

Block 3: Parallel Computer Structures

Exam focus: Flynn's four types with examples, pipelining, vector vs array, UMA/NUMA.

Unit 1: Introduction to Parallel Processing

SLM	Topic	Concepts to cover	Search as
3.1.1	Serial vs parallel	Serial: one processing element (PE) at a time. Parallel: many PEs on same problem or workload; goal = higher throughput or faster	serial vs parallel processing

SLM	Topic	Concepts to cover	Search as
		time-to-solution; need sync and memory sharing rules.	
3.1.2–3	Data processing cycle	Fetch input → process → store/output; in parallel systems, stages or datasets split across PEs; bottlenecks at memory, interconnect, or serial sections (Amdahl idea at intro level).	data processing cycle parallel
3.1.7	Pipeline (intro)	Like assembly line: multiple instructions in different stages at once (IF, ID, EX, MEM, WB); improves throughput ; one instruction still takes full pipeline latency.	pipeline computer architecture intro
3.1.8–9	Array & multiprocessing (intro)	Array processor : many PEs same op on array data. Multiprocessing : several CPUs each with own instruction stream; shared or private memory depending on design.	array processor multiprocessing

Unit 2: Architectural Classification

SLM	Topic	Concepts to cover	Search as
3.2.1–2	Classification overview	Systems classified by # of instruction streams and # of data streams , coupling (loose/tight), memory organisation; helps compare supercomputers, GPUs, clusters.	parallel computer classification
3.2.3	Flynn's taxonomy	SISD : one stream, one data (normal PC). SIMD : one instruction, many data (vector/GPU lanes). MISD : rare (multiple instr on same data). MIMD : many instr, many data	Flynn classification SISD SIMD MIMD

SLM	Topic	Concepts to cover	Search as
		(multiprocessors, clusters). Give one real example each.	
3.2.4	Feng's classification	Classifies by max parallelism in pipeline vs array dimensions; alternative view to Flynn; know it exists and what dimension it emphasises (per SLM).	Feng computer classification
3.2.5	Handler / coupling	Loosely coupled = separate memory/OS nodes. Tightly coupled = shared memory bus. UMA : uniform memory access time. NUMA : local memory faster than remote — affects scheduling and data placement.	UMA NUMA multiprocessor

Unit 3: Pipelining

SLM	Topic	Concepts to cover	Search as
3.3.1–2	Pipelining basics	Divide work into stages ; overlap stages of different instructions; ideal speedup → number of stages if no stalls; latency per instruction vs throughput of pipeline.	instruction pipelining computer
3.3.2	Pipeline hazards	Structural : hardware conflict (one ALU). Data : RAW/WAR/WAW dependencies need forwarding or stall. Control : branches flush wrong-path instructions; branch prediction intro.	pipeline hazards structural data control
3.3.3	Levels of pipelining	Instruction-level (fetch/decode/execute). Arithmetic-level (multiplier pipeline). Processor-level (multiple functional units); each level trades complexity for speed.	levels of pipelining
3.3.4–5	Configuration	Pipelines for fixed-point vs floating-point paths;	superscalar pipeline

SLM	Topic	Concepts to cover	Search as
		superscalar = multiple instructions issued per cycle if no hazards; VLIW idea (compiler schedules).	

Unit 4: Vector Processing and Array Processors

SLM	Topic	Concepts to cover	Search as
3.4.1	Vector processing	Vector registers hold long arrays; one instruction applies same op to all elements (pipelined vector unit); strong for scientific/math kernels; related to SIMD.	vector processing computer
3.4.4	Performance measures	Speedup = $T_{\text{serial}} / T_{\text{parallel}}$. Efficiency = speedup / N processors. Throughput = tasks/time. Utilisation = busy time / total time; know formulas for short answers.	speedup efficiency parallel processing
3.4.5–8	Supercomputers & arrays	Array processors connect many PEs to shared memory or network; supercomputers combine vector, parallel I/O, fast interconnect; CRAY/PARAM-style examples from notes.	array processor architecture

Block 4: Basic Concepts of Operating Systems

Exam focus: boot, layered vs microkernel, system calls, PCB/states, schedulers, FCFS/SJF numericals.

Unit 1: OS Components and Design

SLM	Topic	Concepts to cover	Search as
4.1.1	OS definition	OS = resource manager (CPU, memory, I/O, files) + extended machine (hides hardware	what is operating system

SLM	Topic	Concepts to cover	Search as
		complexity); runs in kernel mode; users run application programs on top.	
4.1.1	Design goals & components	Goals: convenience, efficiency, ability to evolve. Components: process management, memory management, file system, I/O, protection/security, networking/UI as needed.	components of operating system
4.1.3	Booting	Power on → POST/BIOS hardware check → boot loader from disk → load kernel into RAM → init drivers and services → login/shell; know sequence for 4-mark questions.	operating system boot process
4.1.5	Types of OS	Batch (jobs queued). Multiprogramming (several in memory). Time-sharing (interactive slices). Real-time (deadlines). Distributed (networked nodes). Embedded — match features to type.	types of operating system
4.1.6	OS structure	Monolithic: one big kernel. Layered: strict levels (hardware → kernel → services → apps). Microkernel: minimal kernel, services in user space (Mach-style). Modular: loadable kernel modules.	layered OS microkernel

Unit 2: OS Services

SLM	Topic	Concepts to cover	Search as
4.2.1	OS services	Process creation/sync, memory alloc/protection, file read/write, I/O device access, protection (users/permissions), error detection, accounting, networking, GUI/CLI.	operating system services

SLM	Topic	Concepts to cover	Search as
4.2.2	System calls	Programs request kernel via system call (trap); switch user mode → kernel mode ; examples: fork, read, write, open. API (e.g. POSIX) wraps syscalls; API ≠ syscall but related.	system call vs API operating system

Unit 3: Process Scheduling

SLM	Topic	Concepts to cover	Search as
4.3.1–3	Process & states	Program = passive file on disk; process = active instance with PID, memory, registers. States: New → Ready → Running → Waiting → Terminated ; transitions on admit, dispatch, I/O, interrupt, exit. Draw diagram.	process states in operating system
4.3.1–3	PCB	Process Control Block holds PID, state, PC, registers, memory limits, open files, scheduling priority, accounting — kernel uses PCB to suspend/resume process.	process control block PCB
4.3.4–10	Queues & schedulers	Job queue (all jobs), ready queue (in memory, waiting CPU), device queues (waiting I/O). Long-term admits jobs; short-term picks CPU; medium-term swapper controls multiprogramming degree.	long term short term scheduler
4.3.4–10	Context switch	Save old process PCB (registers, PC, state) to memory; load new PCB; pure overhead — no useful user work during switch; frequent with preemptive scheduling.	context switch operating system

SLM	Topic	Concepts to cover	Search as
4.3.4–10	Criteria & formulas	Maximise CPU utilisation and throughput; minimise turnaround, waiting, response time. TAT = Completion time – Arrival time . WT = TAT – Burst time (or start – arrival).	turnaround waiting time scheduling
4.3.4–10	Non-preemptive	Once running, process keeps CPU until it blocks or finishes . FCFS : order of arrival (convoy effect). SJF : shortest burst first among ready — minimises average wait if known.	FCFS scheduling Gantt chart
4.3.4–10	Preemptive	CPU can be taken away on timer, higher priority, or shorter job arrival. SRTF : preempt if new job has shorter remaining time. Round Robin : fixed time quantum , circular ready queue.	preemptive vs non preemptive scheduling
4.3.4–10	FCFS / SJF practice	Draw Gantt chart from AT/BT table; compute per-process WT and average WT ; show idle gaps if CPU free before first arrival; practice SLM/PYQ numeric tables.	SJF scheduling example average waiting time

Unit 4: Multiple Processor Scheduling

SLM	Topic	Concepts to cover	Search as
4.4.1–2	Multiprocessor types	Loosely coupled : separate machines/OS. Tightly coupled : shared memory, common bus. Homogeneous SMP : identical CPUs; scheduling must balance load and avoid contention.	multiprocessor scheduling SMP
4.4.1–2	Asymmetric MP	One master CPU runs OS and I/O; slaves run user code only; simple but master bottleneck; less common now.	asymmetric multiprocessing

SLM	Topic	Concepts to cover	Search as
4.4.1-2	Symmetric MP	Every CPU can run kernel or user; shared ready queue or per-CPU queues; need locking for queue/kernel data structures; standard for modern multicore.	symmetric multiprocessing scheduling
4.4.1-2	Processor affinity	Run thread on same CPU to reuse cache (warm cache) . Soft affinity : OS tries. Hard affinity : process pinned to CPU set (e.g. Linux affinity API concept).	processor affinity operating system

Block 5: Process Synchronization

Exam focus: critical section, Peterson/mutex/semaphore, deadlock four conditions, Banker's algorithm.

Unit 1: Interprocess Communication (IPC)

SLM	Topic	Concepts to cover	Search as
5.1.1	Process structure	Process memory: text (code), data (globals), heap (dynamic alloc), stack (locals, calls); separate address space per process unless shared explicitly.	process structure operating system
5.1.2	States & PCB	Same state diagram as Block 4; emphasise Waiting for IPC/I/O/event; PCB links processes in queues; state change is atomic from scheduler view.	process state diagram OS
5.1.5	IPC models	Shared memory : fastest; processes map same region; need sync (semaphores). Message passing : send/receive messages; copy overhead; good for distributed systems.	IPC shared memory message passing

SLM	Topic	Concepts to cover	Search as
5.1.5	IPC mechanisms	Pipes (anonymous), named pipes (FIFO) , message queues , shared memory segments , semaphores , sockets — know purpose and direction (uni/bi).	inter process communication methods
5.1.6	IPC issues	Race condition : outcome depends on interleaving when shared variable updated by two processes. Leads to need for critical section and mutual exclusion .	race condition critical section

Unit 2: Mutual Exclusion and Synchronization

SLM	Topic	Concepts to cover	Search as
5.2.1–2	Need for sync	Cooperating processes share data or order of execution; without sync, inconsistent results (e.g. two counters, bank balance updates).	process synchronization need
5.2.3	Critical section	Code accessing shared resource = critical section . Need: (1) Mutual exclusion , (2) Progress (someone enters if CS free), (3) Bounded waiting (no infinite postponement). Structure: entry → CS → exit.	critical section problem requirements
5.2.4	Peterson's algorithm	Software solution for 2 processes; uses shared flags <code>turn</code> and <code>interested[]</code> ; satisfies ME/progress/bounded waiting for two threads — know logic for exam sketch.	Peterson algorithm
5.2.4	Hardware & mutex	Test-and-set , compare-and-swap give atomic lock in hardware. Mutex = binary lock; lock before CS, unlock after; only one holder.	mutex vs semaphore

Unit 3: Semaphores and Monitors

SLM	Topic	Concepts to cover	Search as
5.3.1	Semaphores	Integer S ; wait(P) decrements — block if S goes negative; signal(V) increments — wake waiter. Binary (0/1) like mutex. Counting for N identical resources (e.g. N buffer slots).	semaphore wait signal operating system
5.3.2	Monitors	High-level monitor = mutex + condition variables ; only one process active in monitor; wait / signal on conditions; compiler/OS enforces mutual exclusion (Java synchronized idea).	monitor synchronization OS
5.3.x	Classical problems	Producer-consumer (buffer + semaphores). Readers-writers (many readers OR one writer). Dining philosophers (deadlock avoidance with forks) — know problem statement and sync idea.	producer consumer semaphore problem

Unit 4: Deadlock

SLM	Topic	Concepts to cover	Search as
5.4.1-2	Deadlock & 4 conditions	Set of processes blocked forever , each waiting for resource held by another. Four necessary conditions : mutual exclusion, hold and wait, no preemption, circular wait — all four must hold for deadlock.	four conditions of deadlock
5.4.1-2	RAG	Resource allocation graph : processes = circles, resources = squares, edges request/assignment. Cycle in graph $\Rightarrow$ possible deadlock (if single instance per resource type).	resource allocation graph deadlock

SLM	Topic	Concepts to cover	Search as
5.4.3	Handling deadlock	Prevention: break a condition (e.g. ordered resources). Avoidance: never enter unsafe state (Banker's). Detection + recovery: kill or preempt. Ostrich: ignore (most OSes for general deadlock).	deadlock prevention vs avoidance
5.4.3	Banker's algorithm	Given Available , Max , Allocation , compute Need = Max – Allocation . Try allocating to a process only if resulting state is safe (exists safe sequence where all can finish). Step through small matrix example.	bankers algorithm safe state example

Block 6: Memory Management and File Systems

Exam focus: paging vs segmentation, demand paging, FIFO/OPT/LRU numericals, file allocation compare.

Unit 1: Memory Management Strategies

SLM	Topic	Concepts to cover	Search as
6.1.1–2	MM overview	OS must relocate programs, protect processes from each other, allow sharing (libraries), present logical view vs physical RAM layout.	memory management operating system
6.1.1–2	Logical vs physical	CPU generates logical (virtual) address ; MMU maps to physical frame at run time; enables multiprogramming and later paging/virtual memory.	logical vs physical address MMU
6.1.x	Base & limit	Base register = start of partition; limit = size; CPU address checked to lie within [base, base+limit) — simple protection for contiguous allocation.	base limit register memory

SLM	Topic	Concepts to cover	Search as
6.1.8	Swapping	Entire process (or parts) moved RAM ↔ swap space on disk by medium-term scheduler ; frees memory; high context cost; process in swapped-out state not runnable until swapped in.	swapping in operating system
6.1.8	Contiguous allocation	Memory split into partitions (fixed sizes or dynamic holes). Internal fragmentation (unused space inside partition). External fragmentation (free holes too scattered).	contiguous memory allocation
6.1.x	Placement	First-fit : first hole big enough. Best-fit : smallest hole that fits. Worst-fit : largest hole. Next-fit : rotate search; compare fragmentation and speed.	first fit best fit memory allocation

Unit 2: Paging and Segmentation

SLM	Topic	Concepts to cover	Search as
6.2.1-3	Paging	Physical memory = frames ; logical memory = pages of fixed size. Address = (page number, offset). Page table maps page → frame. No external fragmentation ; possible internal in last page.	paging in operating system
6.2.4	Segmentation	User-visible segments (code, stack, heap) variable size. Address = (segment no, offset). Segment table : base + limit per segment. Matches programmer view; external fragmentation possible.	segmentation memory management
6.2.x	Compare	Table compare: fixed vs variable blocks, internal vs external frag, sharing (pages vs whole segments), table structure;	paging vs segmentation

SLM	Topic	Concepts to cover	Search as
		combined paged segmentation idea if in SLM.	
6.2.x	Page fault	Access page with present bit = 0 → trap to OS → load page from disk to free frame → update page table → restart instruction; costly if frequent.	page fault handling

Unit 3: Virtual Memory Management

SLM	Topic	Concepts to cover	Search as
6.3.1–2	Virtual memory	Process sees large logical address space ; only subset in RAM at once; demand paging loads page on first reference; enables running programs larger than physical memory.	virtual memory demand paging
6.3.3	Page replacement	When frames full, pick victim page. FIFO : oldest. Optimal (MIN) : replace page used farthest in future (theoretical best). LRU : least recently used — good practical choice.	page replacement FIFO LRU optimal
6.3.3	Belady's anomaly	For FIFO , increasing number of frames can increase page fault count on some reference strings — know definition and that OPT/LRU don't show this anomaly.	Belady anomaly FIFO
6.3.3	Reference string practice	Given string and 3 frames , fill frame table step-by-step; mark fault/hit; total faults for FIFO, OPT, LRU — match PYQ (e.g. 9,2,3,4,1,2,3,5,1,0,5,9,9,0,1).	page replacement example reference string

Unit 4: File Allocation and Management

SLM	Topic	Concepts to cover	Search as
6.4.1–3	Files & directories	File attributes : name, type, size, location, protection,	file directory structure OS

SLM	Topic	Concepts to cover	Search as
		timestamps. Directory stores metadata. Single-level, two-level (MFD/UFD per user), tree with absolute/relative paths.	
6.4.4	Allocation methods	Contiguous: one block range — fast sequential, external frag, growth hard. Linked: blocks chained by pointers — no external frag, slow random access. Indexed: index block points to data blocks — good direct access, index overhead.	file allocation methods contiguous linked indexed
6.4.4	Compare methods	Exam table: access speed (sequential vs random), fragmentation, reliability if pointer/index lost, suitability for small vs large files; inode as indexed variant on Unix-style systems.	indexed file allocation vs linked

Study order (at a glance)

Block 1 (CA) → Block 2 (I/O) → Block 3 (Parallel) → Block 4 (OS) → Block 5 (Sync) → Block 6 (Memory/Files)

4 units 4 units 4 units 4 units 4 units

4 units

Progress tracker

Block	Title	Units	Studied (date)	Revised (date)
1	Basic Functional Architecture	4		
2	I/O and DMA	4		
3	Parallel Computer Structures	4		

Block	Title	Units	Studied (date)	Revised (date)
4	Basic Concepts of OS	4		
5	Process Synchronization	4		
6	Memory & File Systems	4		

Total: 6 blocks · 24 units

Aligned with SGOU SLM B21CA03DC and CA and OS Note.md.